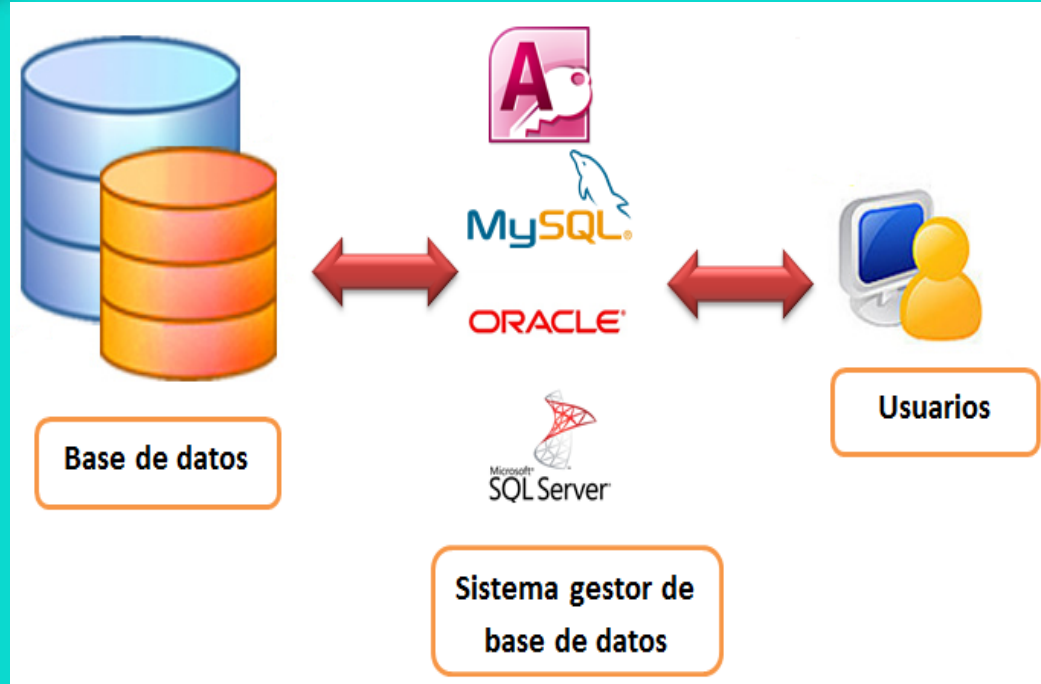




UEA
UNIVERSIDAD
ESTATAL AMAZÓNICA



UNIDAD I:

Fundamentos de Bases de datos relacionales

Tema 2: Principios de bases de datos relacionales.

Subtema: Modelo relacional formal.

RESULTADO DE APRENDIZAJE

Al finalizar el estudio del **Modelo Relacional Formal**, el estudiante será capaz de **identificar, analizar y aplicar los conceptos fundamentales del modelo relacional**, reconociendo relaciones, atributos, tuplas, dominios y claves, así como las restricciones de integridad, para diseñar estructuras de bases de datos relacionales consistentes y representarlas mediante esquemas y tablas que cumplan los principios establecidos por el modelo relacional.

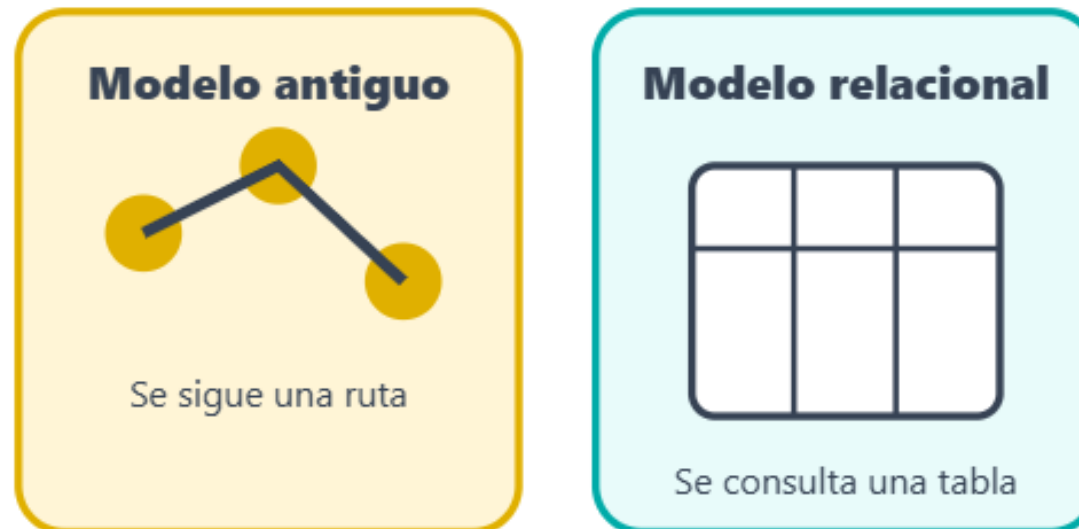
Origen y fundamento del Modelo Relacional

El modelo relacional fue propuesto por **Edgar F. Codd en 1970**. Su objetivo fue separar la forma lógica de ver los datos de la forma física en que se almacenan.

Antes: el programador debía navegar por punteros, listas o jerarquías.

Después: los datos se organizan como relaciones o tablas.

Ventaja: el usuario indica *qué* datos quiere, y el SGBD decide *cómo* obtenerlos.



Modelo Relacional

El **modelo relacional formal** permite representar datos como relaciones. Es el fundamento de bases de datos relacionales modernas y ayuda a diseñar tablas consistentes, sin duplicidad innecesaria y con relaciones correctas.

Los pilares del modelo relacional

R

Relación

Tabla formal

Es una tabla que almacena datos sobre un tema específico. Cada relación tiene un nombre único.

t

Tupla

Fila o registro

Es cada fila de la tabla. Representa una instancia única de la información.

A

Atributo

Columna

Es cada columna de la tabla. Representa una característica o propiedad de los datos.

Una **RELACIÓN** es una tabla formal que almacena datos sobre una entidad o tema específico. Cada relación tiene un nombre único.

Características:

- Es una estructura bidimensional formada por filas (tuplas) y columnas (atributos).
- Representa un conjunto de datos del mundo real (por ejemplo: libros, clientes, empleados).
- Cada relación tiene un nombre que la identifica de manera única dentro de la base de datos.
- Las relaciones se observan en la práctica como TABLAS.



Estructura de una Relación

ATRIBUTOS (COLUMNAS)				TUPLAS (FILAS)
id_libro	título	id_autor	id_editorial	
1	Cien años de soledad	1	1	
2	1984	2	2	
3	El principito	3	1	
...	...	...	...	
GRADO O ARIDAD (número de atributos)			CARDINALIDAD (número de tuplas)	

El **GRADO** es el número de columnas (atributos).
La **CARDINALIDAD** es el número de filas (tuplas o registros).

Ejemplo de Relación

Relación: LIBROS

id_libro	titulo	id_autor	id_editorial
1	Cien años de soledad	1	1
2	1984	2	2
3	El principito	3	1
...	...	...	...



id_libro (PK) Clave primaria: identifica de forma única cada tupla (registro) de la relación.



id_autor(FK) Clave foránea: referencia la clave primaria de otra relación (por ejemplo, AUTORES).



Dominio Conjunto de valores permitidos para cada atributo (por ejemplo, **id_libro** es numérico, titulo es texto).

Una tupla es cada fila o registro de una relación (tabla) en una base de datos relacional. Representa una instancia única de información y está compuesta por un conjunto de valores específicos para cada uno de los atributos (columnas) que conforman la relación.

Características:



ÚNICAS

Cada tupla representa una instancia única de la entidad.



COMPLETAS

Contienen un valor para todos los atributos de la relación.



ORDENADAS

El orden de las tuplas en la tabla puede cambiar, pero cada tupla sigue siendo válida.



INDEPENDIENTES

Las tuplas no dependen del orden o ubicación de otras tuplas.



ATÓMICAS

Cada valor dentro de una tupla es indivisible (no se permiten listas ni grupos repetidos).

Relación: CLIENTES

id_cliente	nombre	email	ciudad
1	Ana Pérez	ana@mail.com	Bogotá
2	Luis Gómez	luis@mail.com	Medellín
3	María López	maria@mail.com	Cali
4	Juan Díaz	juan@mail.com	Barranquilla

← Tupla



Tupla 2: (2, Luis Gómez, luis@mail.com, Medellín)

Es la instancia del cliente con **id_cliente** = 2.

¿ Para qué sirven las Tuplas ?



Almacenan los datos **reales del** negocio (personas, productos, libros, pedidos, etc.).



Permiten registrar múltiples instancias de una misma entidad.



Son la base para consultar, filtrar, ordenar y analizar la información.



Se relacionan con otras tuplas de diferentes tablas mediante claves.

Un **ATRIBUTO** es cada columna de una relación (tabla). Representa una característica o propiedad de la entidad. Cada atributo tiene un nombre único dentro de su relación y un tipo de dato que determina qué valores puede almacenar.

id_libro	titulo	id_autor	id_editorial

↑
Atributo
(columna)

Estructura de los Atributos en una relación

id_libro	titulo	fecha_publicacion	precio	id_autor
INT	VARCHAR(100)	DATE	DECIMAL(10,2)	INT
1	Cien años de soledad	1967-05-30	49.900	1
2	1984	1949-06-08	39.900	2
3	El principito	1943-04-06	29.900	3
...	...	...	...	...

Nombre del atributo
(identifica la columna)

Tipo de dato
(define qué valores puede almacenar)

Cada celda de la columna contiene un valor específico para ese atributo.

Cada atributo debe tener un **nombre único** en la relación.

Relación: LIBROS

id_libro	titulo	fecha_publicacion	precio	id_autor
INT	VARCHAR(100)	DATE	DECIMAL(10,2)	INT
1	Cien años de soledad	1967-05-30	49.900	1
2	1984	1949-06-08	39.900	2
3	El principito	1943-04-06	29.900	3



Cada **columna** (atributo) describe una característica del libro:

- **id_libro**: identificador único del libro.
- **titulo**: nombre del libro.
- **fecha_publicacion**: fecha en que fue publicado.
- **precio**: valor del libro.
- **id_autor**: referencia al autor del libro.

Tipos de Datos para Atributos

123

INT

Números enteros (ej: 1, 2, 100)

\$

DECIMAL(p,s)

Números decimales (ej: 49.90)

ABC

VARCHAR(n)

Texto variable hasta n caracteres (ej: "Cien años")

A

CHAR(n)

Texto fijo de n caracteres (ej: "M")

📅

DATE

Fechas (ej: 2024-05-20)

🔗

BOOLEAN

Valores lógicos (ej: true / false)



Elegir el tipo de dato correcto para cada atributo mejora la integridad y el rendimiento de la base de datos.

Esquema vs Instancia

ESQUEMA

Es la estructura definida en diseño. Indica qué datos se pueden almacenar y cómo se organizan.

Ejemplo

Tabla: ESTUDIANTE

Columna (atributo)	Tipo de dato	Descripción
cedula	CHAR(10)	Identificador del estudiante
nombre	VARCHAR(50)	Nombre del estudiante
email	VARCHAR(80)	Correo electrónico
carrera_id	INT	Identificador de la carrera



El esquema **cambia poco**, porque define la estructura de la tabla.

Esquema vs Instancia

Instancia

Son los datos almacenados en un momento determinado. Representa las filas (registros) que existen en la tabla.

Ejemplo

Tabla: ESTUDIANTE (instancia actual)

cedula	nombre	email	carrera_id
0101	Ana	ana@mail.com	1
0102	Luis	luis@mail.com	1
0103	Carla	carla@mail.com	2
0104	Mario	mario@mail.com	2



La instancia **cambia seguido**, porque los datos se insertan, actualizan o eliminan constantemente.

DIFERENCIAS CLAVE

Aspecto	ESQUEMA	INSTANCIA
 Qué es	Estructura definida en diseño.	Datos almacenados en un momento.
 Qué representa	Tablas, columnas, tipos de datos, relaciones y restricciones.	Filas (registros) con valores concretos.
 Qué tan seguido cambia	Cambia poco.	Cambia seguido.
 Ejemplo	Crear la tabla ESTUDIANTE con sus columnas y tipos de datos.	Insertar estudiantes: Ana, Luis, Carla, Mario, etc.

Propiedades Fundamentales

Son reglas que garantizan que los datos en una base de datos relacional sean consistentes, confiables y fáciles de mantener.



1 Tuplas únicas

No deben existir filas duplicadas.

Ejemplo:

id	nombre	edad
1	Ana	20
2	Luis	21
3	Carla	22
2	Luis	21

✗

id	nombre	edad
1	Ana	20
2	Luis	21
3	Carla	22

✓

No se permiten filas duplicadas.



2 Orden no significativo

El orden de las filas no cambia el significado formal.

Ejemplo:

id	nombre
1	Ana
2	Luis
3	Carla

↔

id	nombre
3	Carla
1	Ana
2	Luis

Ambos representan la misma relación.



3 Atomicidad

Cada celda (atributo) guarda un solo valor.

Ejemplo:

id	telefono
1	0991, 0987

✗

id	telefono
1	0991
2	0987

✓

Cada valor debe ser indivisible.



4 Dominio común

Valores de una columna comparten reglas (dominio).

Ejemplo:

edad
20
21
22
veinte



El dominio de edad es numérico, por lo que todos los valores deben ser números.



Todos los valores de una columna deben pertenecer al mismo dominio.



5 Nombres únicos

No se repiten nombres de atributos en una relación.

Ejemplo:

id	nombre	nombre



id	nombre	email



Cada atributo debe tener un nombre distinto.



6 1FN

La atomicidad inicia la normalización.

Ejemplo:

id	hobbies
1	fútbol, música



id	hobby
1	fútbol
1	música



Los valores deben ser atómicos (uno por celda).

Claves del Modelo Relacional

Las claves son atributos (o conjuntos de atributos) que permiten identificar registros y establecer relaciones entre tablas.

Tipos



1. SUPERCLAVE

Identifica una fila, puede tener atributos sobrantes.

Ejemplo:

En la tabla ESTUDIANTE

cedula	nombre	email	carrera_id
101	Ana	ana@mail.com	1
102	Luis	luis@mail.com	1

{cedula, nombre} también identifica cada fila, pero "nombre" sobra.



2. CLAVE CANDIDATA

Superclave mínima.

Ejemplo:

En la tabla ESTUDIANTE

cedula	email
101	ana@mail.com
102	luis@mail.com

{cedula} y {email} son superclaves mínimas. Ambas son claves candidatas.



3. CLAVE PRIMARIA

Candidata elegida, no admite NULL y debe ser única.

Ejemplo:

En la tabla ESTUDIANTE

cedula	nombre	email	carrera_id
101	Ana	ana@mail.com	1
102	Luis	luis@mail.com	1

Elegimos "cedula" como clave primaria (PK). No puede ser NULL ni repetido.

Tipos de Claves del Modelo Relacional



4. CLAVE ALTERNA

Candidata **no elegida**.

Ejemplo:

En la tabla ESTUDIANTE

cedula	email
101	ana@mail.com
102	luis@mail.com

Si elegimos “cedula” como PK, entonces “email” es clave alterna (candidata no elegida).



5. CLAVE FORÁNEA

Referencia otra tabla (clave primaria o candidata).

Ejemplo:

En la tabla INSCRIPCION

id_inscripcion	estudiante_id	curso_id
1	101	10
2	102	10



“estudiante_id” referencia la PK “cedula” de la tabla ESTUDIANTE.



6. CLAVE COMPUESTA

Usa **varios atributos** para identificar una fila.

Ejemplo:

En la tabla INSCRIPCION

estudiante_id	curso_id	fecha
101	10	2024-05-01
101	11	2024-05-02

La combinación (estudiante_id, curso_id) identifica únicamente cada registro. Esa es la clave compuesta.

La integridad asegura que los datos almacenados cumplan con reglas definidas, evitando errores, duplicados e inconsistencias.

Tipos

- Integridad de entidad
- Integridad referencias
- Integridad de dominio

Tipos de Restricciones de Integridad

1. INTEGRIDAD DE ENTIDAD



Cada entidad debe tener un identificador **único** (clave primaria) que **no puede ser nulo ni repetido**.

Ejemplo:

Tabla: ESTUDIANTE

id_estudiante	nombre	email
1	Ana	ana@mail.com
2	Luis	luis@mail.com
NULL	Carla	carla@mail.com

✗ No se permite NULL ni valores duplicados en la **clave primaria**.

2. INTEGRIDAD REFERENCIAL



Las claves foráneas deben referenciar valores **existentes** en la tabla padre.

Ejemplo:

Tabla padre: CARRERA

id_carrera	nombre_carrera
10	Sistemas
20	Derecho

Tabla hija: ESTUDIANTE

id_estudiante	nombre	id_carrera
1	Ana	10
2	Luis	20
3	Carla	30

i El valor 30 no existe en CARRERA, por lo que viola la **integridad referencial**.

3. INTEGRIDAD DE DOMINIO



Los valores de cada atributo deben pertenecer a un **dominio válido** (tipo de dato, rango, formato, etc.).

Ejemplo:

Tabla: PRODUCTO

id	nombre	precio	
1	Mouse	25.50	✓
2	Teclado	ABC	✗
3	Monitor	-100	✗

i El precio debe ser **numérico** y mayor o igual a 0.

Bibliografía y Recursos Complementarios

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7th ed.). McGraw-Hill Education. Date, C. J. (2004). *Introducción a los Sistemas de Bases de Datos* (8va ed.). Pearson Educación. Codd, E. F. (1970). *A Relational Model of Data for Large Shared Data Banks*. Communications of the ACM, 13(6), 377–387.

Recursos en línea:

PostgreSQL Documentation. Disponible en: <https://www.postgresql.org/docs/>

MySQL Documentation. Disponible en: <https://dev.mysql.com/doc/>

DB-Engines Ranking. Disponible en: <https://db-engines.com/en/>

RESUMEN DE LA SEMANA

El Modelo Relacional Formal, propuesto por Edgar F. Codd en 1970, constituye la base teórica de los sistemas de bases de datos relacionales modernos. Este modelo organiza la información en relaciones (tablas) compuestas por atributos (columnas) y tuplas (filas), permitiendo representar datos de manera estructurada y consistente.

Los conceptos fundamentales incluyen relaciones, atributos, dominios, tuplas, grado y cardinalidad. Además, el modelo establece reglas que garantizan la calidad de los datos mediante claves primarias, claves foráneas, claves candidatas y restricciones de integridad.

La integridad de entidad asegura que cada registro sea único e identificable; la integridad referencial mantiene relaciones válidas entre tablas; y la integridad de dominio garantiza que los valores almacenados cumplan con las reglas definidas para cada atributo.

Estos principios permiten diseñar bases de datos eficientes, reducir redundancia, evitar inconsistencias y facilitar la recuperación de información mediante consultas SQL. Actualmente, sistemas como Oracle Database, PostgreSQL, MySQL y Microsoft SQL Server implementan los fundamentos del modelo relacional para gestionar grandes volúmenes



UEA
UNIVERSIDAD
ESTATAL AMAZÓNICA

JUNTOS TRANSFORMAMOS EL MUNDO DESDE LA AMAZONÍA



**CARRERAS
EN LÍNEA**

***Dr. David Sancho, PhD.
RECTOR UEA***